AI Package Damage Detection System

Image & Video Processing (AIDLL 5052) · Practical No. 8 · Mini Project
Complete explanation notes: what was built, how it works, how to run and demo it, and viva questions with answers

1. The project in 30 seconds

> The system takes a **photo of a cardboard package** and automatically decides **ACCEPT** (intact) or **REJECT** (damaged). For damaged packages it also tells the **type of damage** — *crushed*, *torn* or *water-stained* — draws **boxes around the damage**, and rates the **severity** (Minor / Moderate / Severe). It uses **classical image processing (OpenCV)** to find the box and measure damage, and **two AI models** (a Random Forest and a CNN) whose answers are combined. A **web app** lets you demo it live.

Final results (320 test images): Random Forest **99.69%**, CNN **95.62%**, Ensemble **100%** accuracy.

2. Problem and motivation

- E-commerce and courier hubs (Amazon, Flipkart, Delhivery, Blue Dart…) process thousands of parcels every hour.
- Damaged parcels cause customer complaints, refunds and insurance disputes.
- Manual checking is slow, tiring and inconsistent between workers.
- A camera above the conveyor + this software can inspect every parcel in milliseconds and keep a photo record as evidence.

3. Complete pipeline (big picture)

Input image → Pre-processing → Segmentation → Feature extraction → RF + CNN → Ensemble → Localisation + severity → ACCEPT / REJECT report

Stage	What happens	Techniques (IVP syllabus)
1. Pre-processing	Resize to 128×128, convert colour spaces, remove noise, improve contrast	BGR→HSV / LAB / Gray, Gaussian blur, CLAHE
2. Segmentation	Separate the box from the background and get its outline	Otsu thresholding, morphological opening / closing, contour detection
3. Feature extraction	Convert the image into 28 numbers that describe damage	Convex hull, convexity defects, solidity, polygon approximation, Canny, Laplacian, connected components, LBP, colour statistics, illumination correction
4. Classification	Predict intact / crushed / torn / stained	Random Forest (features) + CNN (pixels) → average of probabilities
5. Localisation	Find where the damage is and how big	Thresholding, morphology, contours, bounding boxes, convexity defects
6. Output	Decision, type, confidence, severity, annotated image	Drawing functions (rectangle, contour, text)

4. Project files — what each one is

File / folder	Purpose
`Package_Damage_Detection.ipynb`	Main Jupyter notebook: full code, explanations, graphs and results. **This is the submission.**
`Package_Damage_Detection.pdf`	PDF export of the notebook (report item 1).
`damage_detector.py`	The same model code as a Python module, used by the app (generator, segmentation, features, CNN, localisation, `inspect_package()`, training and loading).
`models/`	Saved trained models: `random_forest.joblib` and `cnn.pt`. The app loads them so it starts instantly.
`app.py`	The Streamlit web application (live demo).
`run_app.bat`	Double-click to start the app.
`export_pdf.py`	Re-runs the notebook and rebuilds the PDF.
`Package_Damage_Detection.html`	Temporary HTML used to make the PDF (not runnable).

5. Dataset — synthetic package images

Real labelled photos of damaged parcels are rare, so the project includes a **procedural image generator** (`make_package()`) that draws realistic packages. 400 images per class are generated → **1,600 images**.

Random variation (so the model generalises)

- Box size, position, rotation (±15°), perspective
- Cardboard colour, corrugation texture, paper grain
- Lighting gradient, exposure (±15%), blur, noise
- Background grey level, conveyor-belt stripes

Distractors (must NOT be called damage)

- Packing tape strip (lighter band)
- White shipping label with barcode
- Printed "this side up" arrows

Class	How the damage is drawn	What reveals it
intact	Clean box with tape / label / arrows only	Convex outline, uniform brightness
crushed	One corner pushed inwards + dark crease lines + shadow	Concave outline (low solidity, deep convexity defect), extra edges
torn	1–2 irregular dark holes with a light frayed border	Very dark blobs inside the box
stained	1–2 blurred dark blotches with a tide-line ring; faint to strong	Moderate brightness drop over a large area

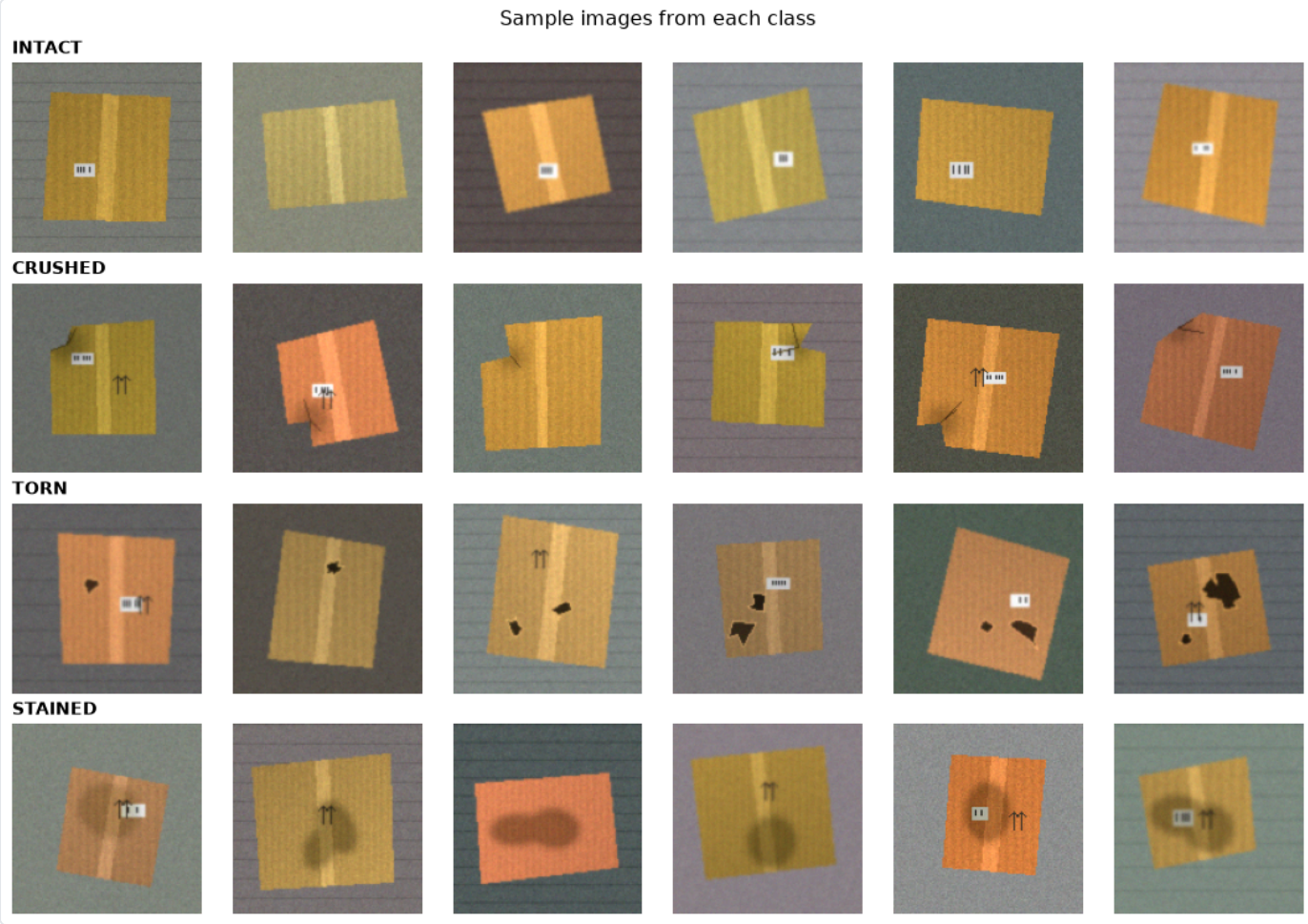


Figure 1 – Sample generated images for each class

Split: **80% training (1,280)**, **20% testing (320)**, stratified so each class has 80 test images.

6. Image processing — step by step

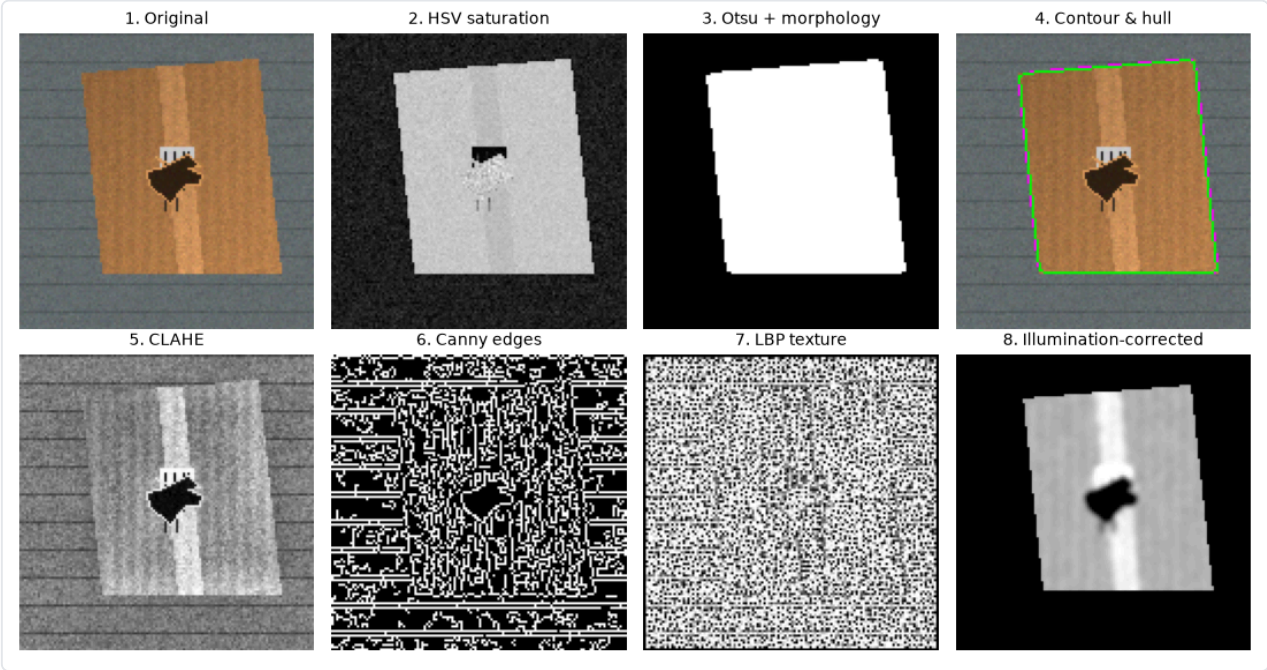


Figure 2 – The 8 processing steps on a torn package (same view as the app's "See the image-processing steps")

Step 1 – Colour-space conversion

OpenCV loads images as **BGR**. The project uses three other spaces:

- **HSV** (Hue, Saturation, Value) – S separates coloured cardboard from grey background; V measures brightness (used for holes).
- **LAB** – L is perceptual lightness (used for stains), independent of colour.
- **Grayscale** – for edges, texture and Laplacian.

Step 2 – Gaussian blur

Smoothing with a Gaussian kernel (5×5) removes pixel noise so thresholding gives clean regions.

Step 3 – Otsu thresholding

Converts the saturation image to black/white. Otsu's method automatically finds the threshold that minimises the variance inside the two groups (box vs background). No manual threshold needed, so it adapts to each image.

Step 4 – Morphological operations

- **Opening** (erode → dilate): removes small white noise specks.
- **Closing** (dilate → erode), 2 iterations: fills small gaps and holes.
- **Erosion** of the final mask by 5×5: ignores the box border when measuring the inside.

Step 5 – Contour detection and package mask

`cv2.findContours` finds the outlines; the **largest contour** is the package. Filling it gives the package mask (labels and holes inside are included). From the contour we compute the **convex hull** (magenta in Fig. 2).

Step 6 – CLAHE and Canny edges

CLAHE boosts local contrast in 8×8 tiles (clip limit 2). **Canny** (thresholds 60 / 150) then finds edges: creases and torn borders increase *edge density*.

Step 7 – LBP texture

Each pixel is compared with its 8 neighbours → an 8-bit code. Codes are grouped into 10 rotation-invariant "uniform" bins; the normalised histogram describes surface texture.

Step 8 – Illumination correction

Real lighting is uneven (one side brighter). A plane `L = a·x + b·y + c` is fitted to the package lightness with **least squares** (fitted twice, ignoring outliers like stains and labels) and the image is divided by it. After this, a value of 1.0 means "normal brightness", so stains (0.55–0.88) are detected reliably anywhere on the box.

7. The 28 features

Group	Feature	Meaning / why it helps
Shape (7)	area_ratio	Package area / image area
	solidity	Contour area / convex-hull area. ≈1 for a normal box, lower when crushed
	extent	Contour area / minimum rotated rectangle area
	n_vertices	Corners after polygon approximation (approxPolyDP). 4 for a box, more when crushed
	circularity	$4\pi \cdot \text{area} / \text{perimeter}^2$
	max_defect	Deepest convexity defect (dent depth), normalised by box size
	sum_defect	Total depth of all significant defects
Darkness (2)	dark_ratio	Fraction of pixels darker than 50% of median V
	dark_blob	Area of the largest dark connected component → holes / tears
Stain (1)	**stain_ratio**	Fraction of pixels at 55–88% of normal brightness after illumination correction
Edges (2)	edge_density	Fraction of Canny edge pixels inside the box
	laplacian_var	Variance of the Laplacian (amount of sharp detail)
Colour (6)	H, S, V mean and std	Overall colour and how uneven it is
Texture (10)	lbp_0 … lbp_9	Uniform LBP histogram

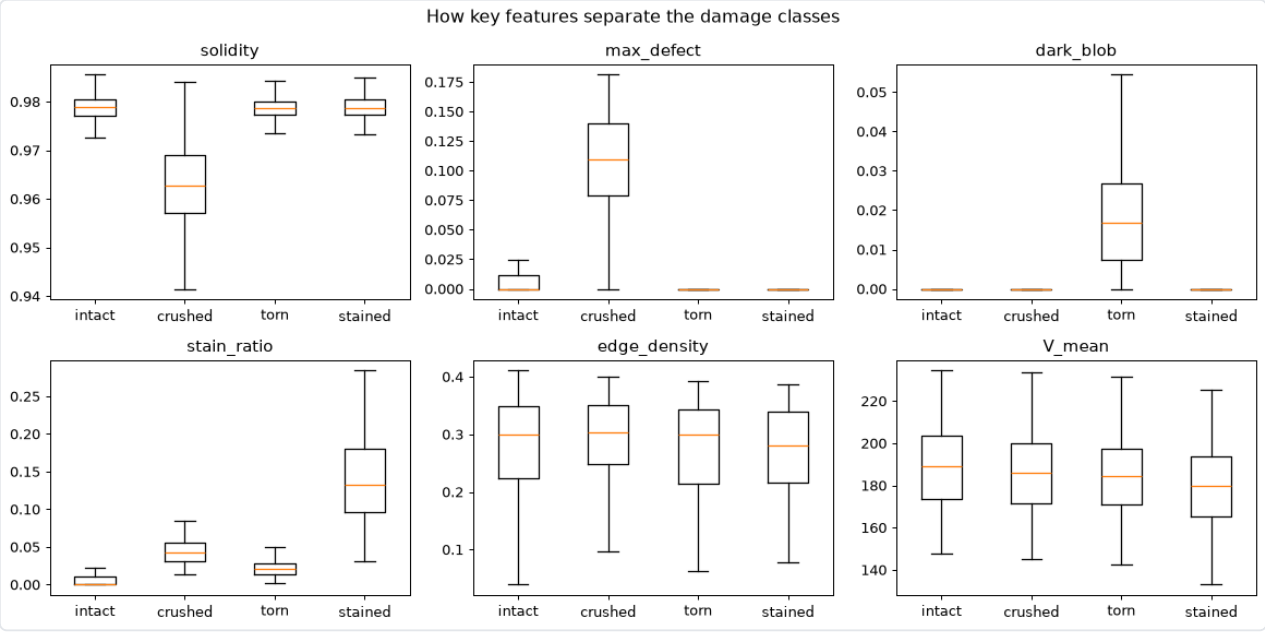


Figure 3 – Key features clearly separate the classes (e.g. low solidity = crushed, high dark_blob = torn, high stain_ratio = stained)

8. AI models

Model 1 – Random Forest (machine learning on features)

- 300 decision trees, each trained on a random bootstrap sample and random subsets of features.
- Prediction = average of the trees' votes → class probabilities.
- Advantages: fast, very accurate on good features, explains which features matter (feature importance).

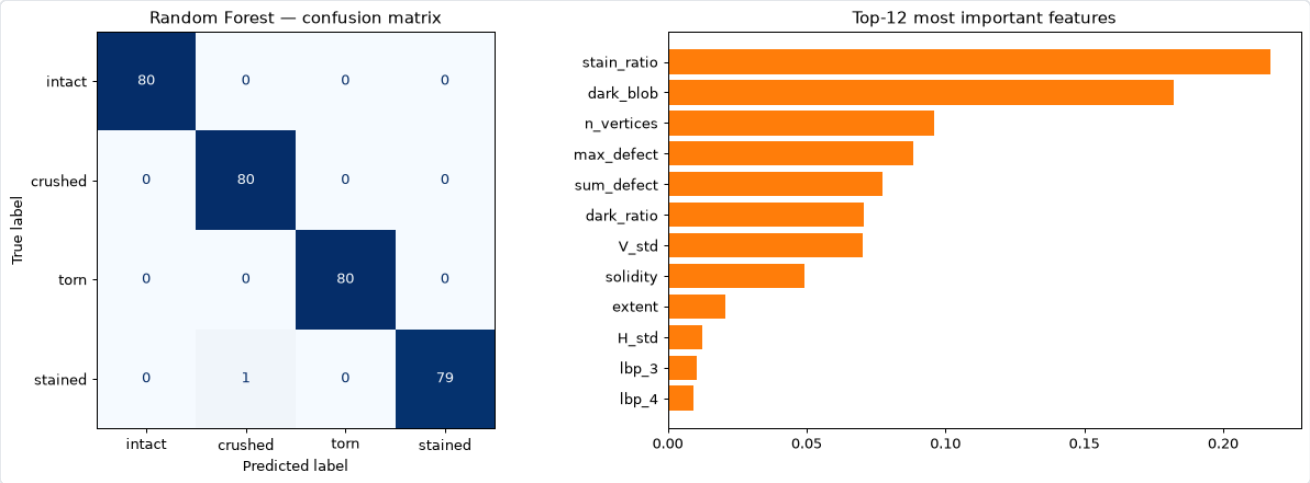


Figure 4 – Random Forest confusion matrix and most important features (stain_ratio and dark_blob are the top features)

Model 2 – Convolutional Neural Network (deep learning on pixels)

```
Input 3 x 128 x 128 (RGB)
 -> [Conv 3x3, 16]  -> BatchNorm -> ReLU -> MaxPool 2x2     (64 x 64)
 -> [Conv 3x3, 32]  -> BatchNorm -> ReLU -> MaxPool 2x2     (32 x 32)
 -> [Conv 3x3, 64]  -> BatchNorm -> ReLU -> MaxPool 2x2     (16 x 16)
 -> [Conv 3x3, 64]  -> BatchNorm -> ReLU -> MaxPool 2x2     (8 x 8)
 -> Global Average Pooling (64) -> Dropout 0.3 -> Fully Connected (4) -> Softmax
```

Term	Simple meaning
Convolution	A small 3×3 filter slides over the image and detects a pattern (edge, corner, texture). The network learns the filter values.
BatchNorm	Normalises layer outputs so training is faster and stable.
ReLU	Activation: max(0, x). Adds non-linearity.
MaxPool	Keeps the strongest value in each 2×2 block; halves the size, adds position tolerance.
Global Avg Pool	Averages each feature map to one number → few parameters, less over-fitting.
Dropout 0.3	Randomly switches off 30% of neurons during training to prevent over-fitting.
Softmax	Turns the 4 outputs into probabilities that sum to 1.
Cross-entropy loss	Measures how wrong the predicted probabilities are; training minimises it.
Adam + One-Cycle	Optimiser that updates the weights; the learning rate rises then falls (max 0.003) for faster, smoother training.
Epoch / batch	25 epochs (passes over the data), 64 images per batch.
Augmentation	Random horizontal / vertical flips create more variety from the same data.

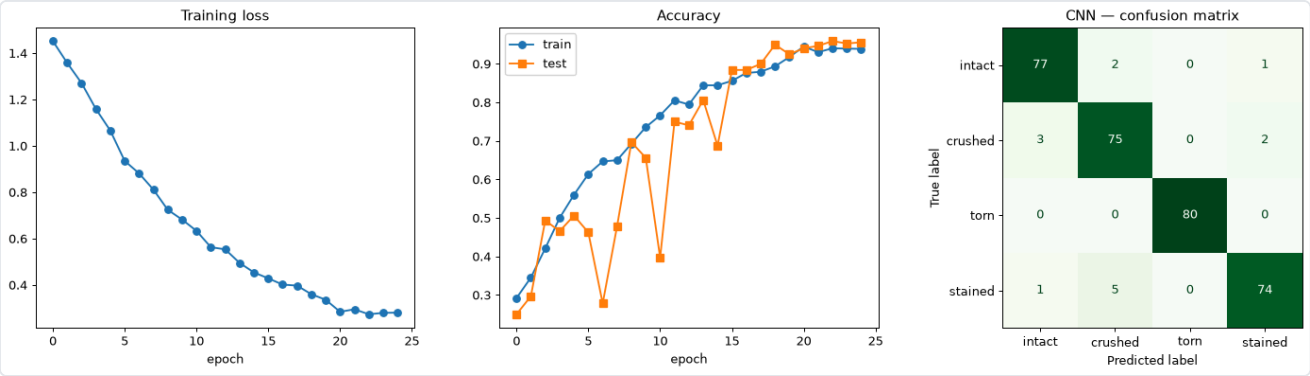


Figure 5 – CNN training loss, accuracy per epoch and confusion matrix

Ensemble

> **P(final) = 0.5 × P(Random Forest) + 0.5 × P(CNN)**. The class with the highest probability is the answer; its probability is the **confidence**. If the class is *intact* → ACCEPT, otherwise REJECT.

9. Evaluation and results

Model	Accuracy (4 classes)	Notes
Random Forest (28 features)	**99.69%**	1 stained box predicted as crushed
CNN (raw pixels)	**95.62%**	Mostly confuses crushed ↔ intact and stained
Ensemble (RF + CNN)	**100%**	All 320 test images correct

Binary damage detection (ensemble): accuracy 100%, precision 100%, recall 100% — 0 damaged parcels missed, 0 good parcels wrongly rejected.

Metric	Formula	Meaning
Accuracy	(TP + TN) / all	Overall fraction correct
Precision	TP / (TP + FP)	Of the rejected boxes, how many were really damaged
Recall	TP / (TP + FN)	Of the damaged boxes, how many were caught (most important here)
F1-score	2·P·R / (P + R)	Balance of precision and recall

> **Be honest in the viva:** these scores are on synthetic test images generated the same way as the training images. On real photographs accuracy would be lower until the models are retrained on real data.

10. Damage localisation and severity

Damage	How it is located	Box colour
Tear / hole	Pixels with V < 50% of the median → opening 3×3 → contours with area ≥ 12 px → bounding boxes	Red
Crush / dent	Convexity defects deeper than 5% of the box size; the area around a dent is excluded from stain detection (its shadow is not a stain)	Orange
Stain	Illumination-corrected lightness 0.55–0.88 → remove hole areas → opening 7×7 → contours ≥ 100 px	Blue

Severity = damaged area ÷ package area: **Minor** < 3%, **Moderate** 3–8%, **Severe** > 8%. The package outline is drawn **green** for ACCEPT and **red** for REJECT.

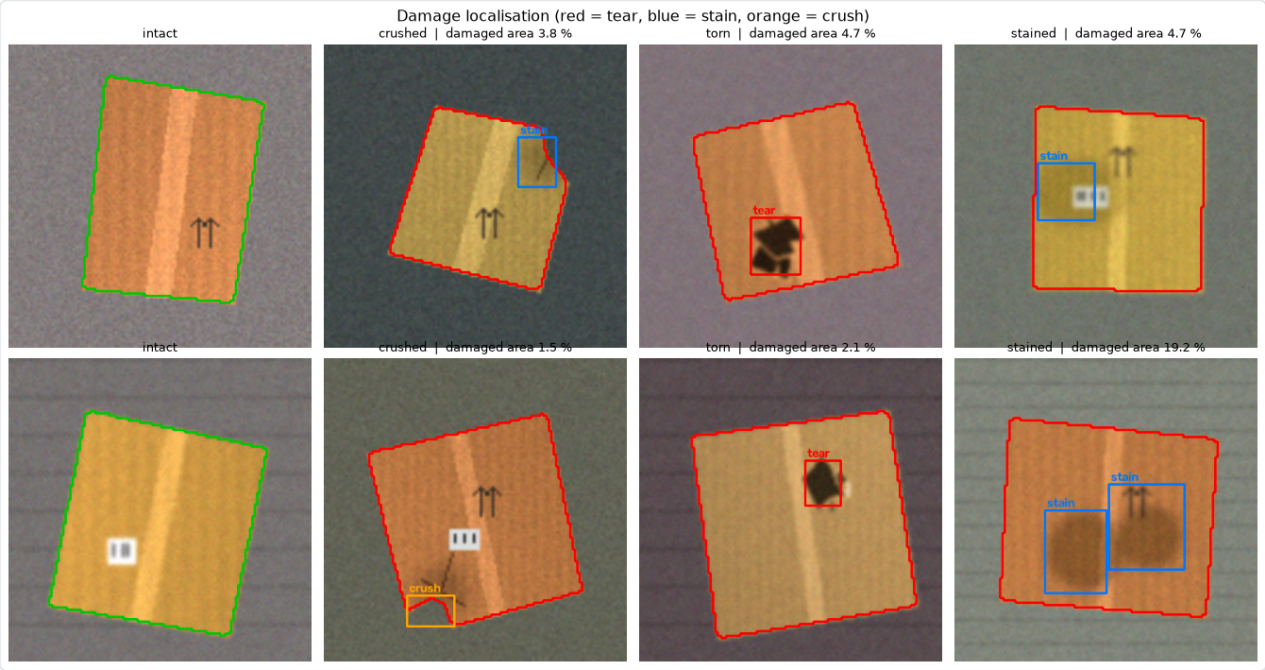


Figure 6 – Damage localisation (red = tear, blue = stain, orange = crush)

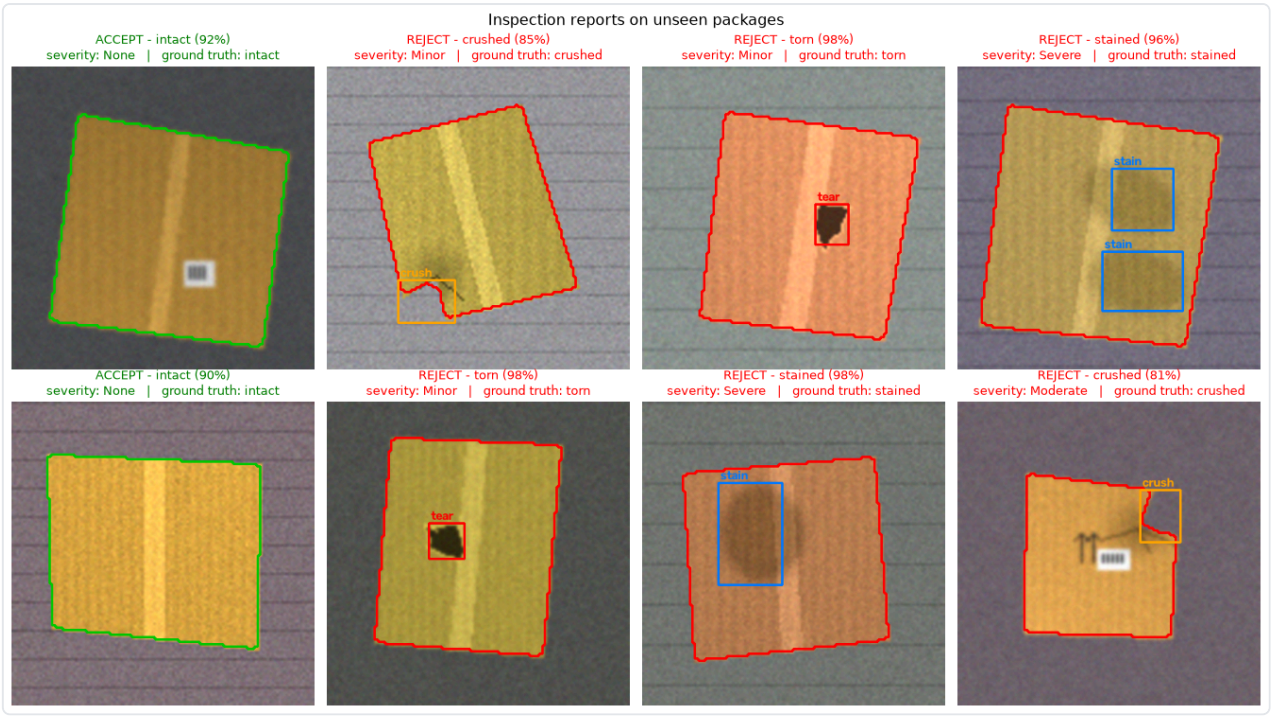


Figure 7 – Full inspection reports on packages the models have never seen

11. How the web app works

The app (`app.py`) is built with **Streamlit**, a Python library that turns a script into a web page. Every time you click a button, Streamlit re-runs the script and redraws the page.

What happens when you click "New package"

1. At start-up, `get_models()` loads the saved Random Forest and CNN from `models/`. It is wrapped in `@st.cache_resource`, so models load **only once**, not on every click. (If the files are missing, it trains them, ~1 min.)
2. The button increases a counter in `st.session_state`; this counter is the random seed, so you get a new package.
3. `make_package(type, rng)` creates the image (or the uploaded / camera image is decoded with `cv2.imdecode`).
4. `inspect_package(img)` runs: resize → segmentation → 28 features → RF probabilities + CNN probabilities → average → localisation → severity → annotated image.
5. The page shows the green/red verdict, 4 metric cards, the probability bar chart, the annotated image, and an expander with the 8 processing steps.

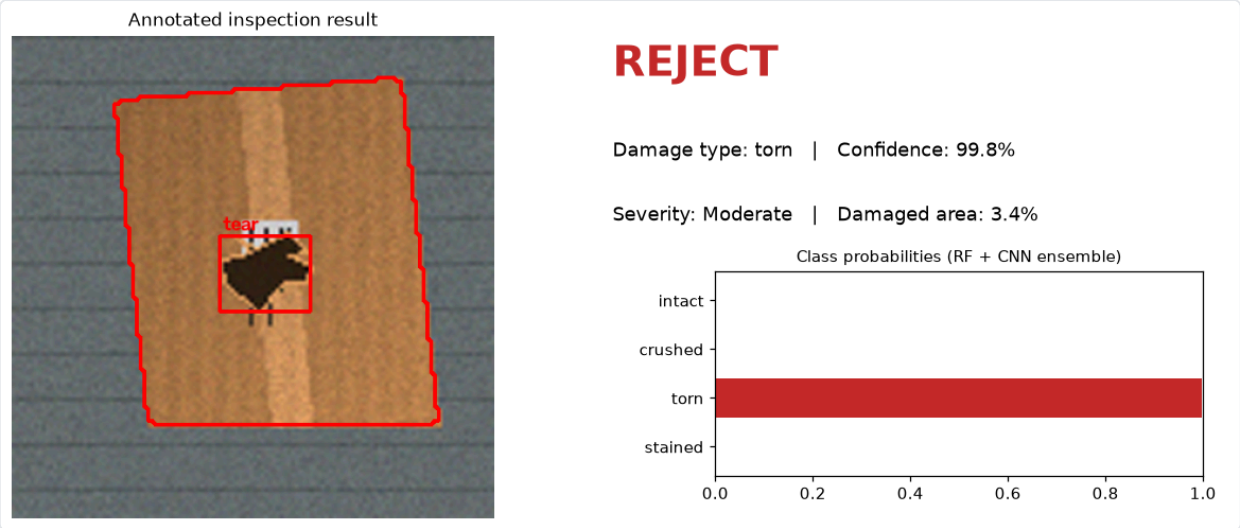


Figure 8 – What the result panel of the app shows for a torn package

The three tabs

Tab	What it does
Inspect a package	Choose the source: **Generate test package** (pick a damage type or random), **Upload photo**, or **Camera**. Shows the full report and the processing steps.
Conveyor belt simulation	Inspects a batch of 4–16 random packages like a sorting line. Shows counts of inspected, accepted, rejected and correctly classified packages, and a card for each package with its actual type.
How it works	Short summary table of the techniques and accuracy.

12. How to run everything

Task	How
Start the demo app	Double-click `run_app.bat` → the browser opens `http://localhost:8501`. Keep the black window open; close it to stop.
Start app (terminal)	`py -3.14 -m streamlit run app.py`
Run the notebook in VS Code	Open the `.ipynb` → Select Kernel → Python 3.14 → Run All (results appear under each cell; ~2 min)
Rebuild the notebook PDF	`py -3.14 export_pdf.py` (after editing name / roll no.)
Retrain the app models	`py -3.14 damage_detector.py` (~70 s)

13. Demo script for ma'am (about 3 minutes)

1. **Introduce** (20 s): "This is an AI package damage detection system. A camera image of a parcel goes in; the system says accept or reject, the damage type, where it is and how severe."
2. **Inspect tab**: select *intact* → New package → green ACCEPT. Then *torn*, *crushed*, *stained* → red REJECT with coloured boxes. Point at confidence and severity.
3. **Open "See the image-processing steps"**: explain HSV saturation → Otsu → contour and convex hull → CLAHE → Canny → LBP → illumination correction. (This shows the IVP concepts.)
4. **Conveyor tab**: 8 or 12 packages → Run conveyor. "This is how it would work on a sorting line."
5. **Results**: "Random Forest 99.7%, CNN 95.6%, the ensemble got 100% on 320 test images."
6. **Limitation + future**: "It is trained on synthetic images; with a real dataset and YOLO it could handle real conveyor video."

14. Limitations and future scope

Limitations

- Trained on synthetic images; real photos look different
- One package per image, top face only
- Assumes brown cardboard on a less colourful background
- Low resolution (128×128) misses very small damage

Future scope

- Collect and train on real parcel photos
- YOLO / object detection for many boxes at once
- Live video from the conveyor camera, frame by frame
- Multiple cameras for all 6 faces; robotic arm to remove rejects

15. Viva questions and answers

Q1. What is your project?

> An AI system that looks at an image of a package and decides ACCEPT or REJECT. If it is damaged, it tells the type of damage (crushed, torn or water-stained), where the damage is, and how severe it is.

Q2. Why did you choose this topic?

> Warehouses and courier companies handle thousands of parcels per hour. Checking each box by hand is slow, tiring and inconsistent. Automatic visual inspection is a real industrial use of image processing.

Q3. Is it image processing or video processing?

> Image processing. Each package is one image. It can be extended to video by running the same function on every frame from a conveyor-belt camera.

Q4. Which dataset did you use?

> A synthetic dataset of 1,600 images (400 per class) that my code generates. Real labelled photos of damaged parcels are hard to find, so the generator draws cardboard boxes with random size, rotation, colour, texture, lighting, noise, tape, labels and arrows, and adds damage. The code automatically uses a real dataset if one is placed in the dataset/ folder.

Q5. Why synthetic data? Is that cheating?

> No. Synthetic data is commonly used when real data is rare (for example in self-driving and medical AI). It also gives perfect labels. The limitation is that real photos look different, so for real deployment the model must be retrained on real images. I mention this honestly in the conclusion.

Q6. What are the 4 classes?

> intact (no damage), crushed (corner pushed in, creases), torn (holes / rips), stained (water or liquid stains).

Q7. What libraries did you use?

> OpenCV (image processing), NumPy (arrays), scikit-learn (Random Forest and metrics), PyTorch (CNN), Matplotlib (plots), Streamlit (web app), Jupyter (notebook).

Q8. How do you separate the package from the background?

> Cardboard is a strongly coloured (saturated) material while floors and conveyor belts are grey. So I convert the image to HSV, take the Saturation channel, blur it, and apply Otsu thresholding. Then morphological opening and closing clean the mask, and the largest contour is the package.

Q9. What is Otsu thresholding?

> An automatic method to choose a threshold. It tries every threshold value and picks the one that best separates the histogram into two groups (minimum within-class variance). So I don't have to choose the threshold by hand.

Q10. Why HSV and not RGB?

> In RGB, colour and brightness are mixed. HSV separates Hue (colour), Saturation (purity) and Value (brightness). Saturation alone separates brown cardboard from grey background even when lighting changes.

Q11. What are morphological operations?

> Operations on the shape of binary images using a small kernel. Erosion shrinks white regions, dilation grows them. Opening = erosion then dilation (removes small white noise). Closing = dilation then erosion (fills small holes and gaps).

Q12. How do you detect a crushed corner?

> A normal box outline is convex (a quadrilateral). A crushed corner makes the outline cave inwards. I compute the convex hull and the convexity defects; a deep defect means a dent. Solidity = contour area / hull area also drops below 1 for a crushed box.

Q13. What is a convex hull and convexity defect?

> The convex hull is the smallest convex shape enclosing the contour (like a rubber band around it). A convexity defect is a place where the contour goes inside the hull; its depth tells how deep the dent is.

Q14. How do you detect tears / holes?

> A hole shows the dark inside of the box. Pixels darker than 50% of the package's median brightness (V channel) are marked, cleaned with morphological opening, and connected components larger than 12 pixels are tears.

Q15. How do you detect water stains?

> Stains are moderately darker regions. Because lighting may be uneven, I first fit a plane to the brightness of the package using least squares and divide by it (illumination correction). Regions that are then 55-88% as bright and large enough are stains.

Q16. What is CLAHE?

> Contrast Limited Adaptive Histogram Equalisation. It improves contrast locally in small tiles instead of the whole image, and limits the amplification so noise is not boosted too much. I use it before edge detection.

Q17. What is Canny edge detection?

> A multi-step edge detector: Gaussian smoothing, gradient calculation (Sobel), non-maximum suppression to thin edges, and double thresholding with hysteresis (60 and 150 here). Edge density is higher on creased or torn boxes.

Q18. What is LBP?

> Local Binary Pattern, a texture descriptor. Each pixel is compared with its 8 neighbours: neighbour >= centre gives 1, else 0, making an 8-bit code. I use the rotation-invariant 'uniform' version with 10 bins. The histogram describes surface texture.

Q19. What is the Laplacian variance?

> The Laplacian is a second-derivative filter that responds to rapid intensity changes. Its variance measures how much fine detail or sharp change is present (creases and torn edges increase it).

Q20. How many features and which ones?

> 28 features: 7 shape (area ratio, solidity, extent, number of polygon vertices, circularity, max and total defect depth), 2 darkness (dark ratio, largest dark blob), 1 stain ratio, 2 edge (Canny density, Laplacian variance), 6 colour (mean and std of H, S, V) and 10 LBP texture bins.

Q21. What is a Random Forest?

> An ensemble of many decision trees (300 here). Each tree is trained on a random sample of the data and random features; the final answer is the majority vote / average probability. It is accurate, resists over-fitting and tells feature importance.

Q22. What is a CNN?

> A Convolutional Neural Network learns its own features from pixels. Convolution layers slide small filters over the image to detect edges, then textures, then shapes. My CNN has 4 blocks of Conv 3x3 + BatchNorm + ReLU + MaxPool (16, 32, 64, 64 filters), then Global Average Pooling, Dropout 0.3 and a fully connected layer with 4 outputs.

Q23. How was the CNN trained?

> Input 128x128 RGB, loss = cross-entropy, optimiser = Adam with a One-Cycle learning-rate schedule (max 0.003), batch size 64, 25 epochs, augmentation with random horizontal and vertical flips. Training takes about 1 minute on a CPU.

Q24. Why use two models? What is the ensemble?

> They make different kinds of mistakes: the Random Forest uses hand-made image-processing features, the CNN learns from pixels. I average their class probabilities: P = 0.5 x P_RF + 0.5 x P_CNN. The ensemble reached 100% test accuracy, better than the CNN alone.

Q25. How did you evaluate the model?

> 80% training / 20% testing split (stratified, 320 test images). I report accuracy, precision, recall, F1-score and the confusion matrix for each model.

Q26. What are precision and recall?

> Precision = of the packages the model rejected, how many were really damaged (TP / (TP + FP)). Recall = of all damaged packages, how many the model caught (TP / (TP + FN)). For inspection, recall is very important because a missed damaged parcel reaches the customer.

Q27. What is a confusion matrix?

> A table of true class vs predicted class. Diagonal values are correct predictions, off-diagonal values show which classes are confused (for example the CNN sometimes confuses crushed and intact).

Q28. What were your results?

> Random Forest 99.69%, CNN 95.62%, Ensemble 100% on the 320 test images. Binary damage detection: 100% precision and 100% recall on the test set.

Q29. How is severity calculated?

> Damaged area (holes + stains + dent region) as a percentage of the package area: Minor < 3%, Moderate 3-8%, Severe > 8%.

Q30. How does the app work?

> It is a Streamlit web app (app.py). It loads the trained models once (cached), takes an image (generated, uploaded or from the camera), calls inspect_package() from damage_detector.py, and shows the verdict, metrics, probability chart, annotated image and all processing steps.

Q31. Will it work on real photos?

> Partly. The image-processing steps work on real images, but the AI models were trained on synthetic images, so accuracy on real photos will be lower. Retraining with real photos (put in dataset/) would fix this.

Q32. What are the limitations?

> Trained on synthetic data; only one package per image; only the visible face is inspected; assumes a brown box on a less colourful background; low resolution (128x128).

Q33. What is the future scope?

> Train on a real dataset, use object detection (YOLO) for many packages at once, process live conveyor video frame by frame, use multiple cameras to see all faces, and connect to a robotic arm that removes rejected boxes.

16. Quick revision sheet

Item	Value
Classes	intact, crushed, torn, stained
Dataset	1,600 synthetic images (400 / class), 128×128, split 80 / 20
Segmentation	HSV saturation → Gaussian 5×5 → Otsu → open + close → largest contour
Features	28 (7 shape, 2 dark, 1 stain, 2 edge, 6 colour, 10 LBP)
Random Forest	300 trees → 99.69%
CNN	4 conv blocks (16-32-64-64), GAP, dropout 0.3; Adam + One-Cycle, 25 epochs, batch 64 → 95.62%
Ensemble	average of probabilities → 100%
Hole threshold	V < 50% of median
Stain range	0.55–0.88 of illumination-corrected lightness
Dent threshold	convexity defect depth > 5% of box size
Severity	Minor < 3%, Moderate 3–8%, Severe > 8% damaged area
Canny thresholds / CLAHE	60, 150 / clip 2.0, tiles 8×8
Tools	Python, OpenCV, NumPy, scikit-learn, PyTorch, Matplotlib, Streamlit, Jupyter